Write a script that will review event logs. It will

allow the user to select which event log to analyze:

The program has to

- Read Application, System, Security, Setup, and Windows PowerShell logs.
- Allow the user to select one log or all five.
- Display the event timestamp, Event ID, source, severity, and computer name.
- Export the results to CSV for analysis in Excel or Pandas.
- Continue processing other logs if one log cannot be accessed.
- Work with Python 3.11 and Spyder on Windows.

For this assignment, I recommend using **Python 3.11 with the `pywin32` library** in Spyder 6. This library allows Python to communicate with Windows Event Viewer and retrieve event logs directly from your computer.

The script below includes all five required Windows event logs, lets the user select individual logs or all five, displays the requested information, and exports the results to a CSV file.

1. Install the required library

Since you're using Spyder on Windows, open **Command Prompt (CMD)** rather than the Spyder console.

Run:

```bash
py -3.11 -m pip install pywin32
```

Wait for the installation to finish.

You only need to install this library once.

2. Python script – Windows Event Log Analyzer

Create a new Python file in Spyder and name it:

`event_log_analyzer.py`

Copy the following code into Spyder.

python

```python
import win32evtlog
import csv
from datetime import datetime


# Available Windows Event Logs
EVENT_LOGS = {
    "1": "Application",
    "2": "System",
    "3": "Security",
    "4": "Setup",
    "5": "Windows PowerShell"
}


# Convert Windows event severity into readable text
def get_severity(event_type):

    severity = {
        1: "Error",
        2: "Warning",
        4: "Information",
        8: "Audit Success",
        16: "Audit Failure"
    }

    return severity.get(event_type, "Unknown")


# Display menu
def display_menu():

    print("\n=============================================")
    print("          WINDOWS EVENT LOG ANALYZER")
    print("=============================================")

    print("\nSelect an event log to analyze:\n")

    for number, log_name in EVENT_LOGS.items():
        print(f"{number}. {log_name}")
```

```
    print("6. All Event Logs")

    print("=============================================")


# Read selected event log
def read_event_log(log_name):

    events = []

    print(f"\nAnalyzing {log_name} Log...")
    print("-" * 60)

    try:

        # Open Windows Event Log
        handle = win32evtlog.OpenEventLog(None, log_name)

        flags = (
            win32evtlog.EVENTLOG_SEQUENTIAL_READ |
            win32evtlog.EVENTLOG_FORWARDS_READ
        )

        while True:

            records = win32evtlog.ReadEventLog(
                handle,
                flags,
                0
            )

            # Stop when there are no more events
            if not records:
                break

            for event in records:

                event_data = {
                    "Log Name": log_name,
                    "Timestamp": str(event.TimeGenerated),
                    "Event ID": event.EventID & 0xFFFF,
                    "Source": event.SourceName,
                    "Severity": get_severity(event.EventType),
```

```
                    "Computer Name": event.ComputerName
                }

                events.append(event_data)

                # Display event information
                print("\nTimestamp:", event_data["Timestamp"])
                print("Event ID:", event_data["Event ID"])
                print("Source:", event_data["Source"])
                print("Severity:", event_data["Severity"])
                print("Computer Name:", event_data["Computer Name"])
                print("-" * 60)

        win32evtlog.CloseEventLog(handle)

        print(f"\nSuccessfully processed {log_name} Log.")
        print(f"Total events found: {len(events)}")

    except Exception as error:

        print(f"\nUnable to access {log_name} Log.")
        print("Error:", error)

        print("Continuing with the next event log...")

    return events


# Export results to CSV
def export_to_csv(events):

    if not events:
        print("\nNo events available to export.")
        return

    filename = (
        "Event_Log_Report_"
        + datetime.now().strftime("%Y%m%d_%H%M%S")
        + ".csv"
    )

    try:

        with open(filename, "w", newline="", encoding="utf-8") as file:
```

```
            fieldnames = [
                "Log Name",
                "Timestamp",
                "Event ID",
                "Source",
                "Severity",
                "Computer Name"
            ]

            writer = csv.DictWriter(
                file,
                fieldnames=fieldnames
            )

            writer.writeheader()
            writer.writerows(events)

        print("\n==============================================")
        print("             EXPORT COMPLETE")
        print("==============================================")

        print("File saved as:", filename)
        print("Total records exported:", len(events))

    except Exception as error:

        print("\nError exporting CSV file:", error)


# Main program
def main():

    display_menu()

    choice = input("\nEnter your selection (1-6): ").strip()

    all_events = []

    # Process selected event log
    if choice in EVENT_LOGS:

        selected_log = EVENT_LOGS[choice]
```

```
            all_events.extend(read_event_log(selected_log))

        # Process all five event logs
        elif choice == "6":

            for log_name in EVENT_LOGS.values():

                all_events.extend(read_event_log(log_name))

        else:

            print("\nInvalid selection. Please choose 1-6.")
            return

        # Export collected events
        export_to_csv(all_events)

        print("\nEvent Log Analysis Complete!")


# Run program
if __name__ == "__main__":
    main()
```

---

3. Understanding what the script does

Here is how the different sections of the program work.

Function	Purpose
`display_menu()`	Displays the five available Windows event logs and the option to analyze all five.
`get_severity()`	Converts Windows event severity numbers into readable descriptions.
`read_event_log()`	Opens and reads the selected Windows event log.
`export_to_csv()`	Saves collected event information into a CSV file.
`main()`	Controls the program and processes the user's selection.

Event logs being analyzed

The program supports all five logs required by your assignment.

Event Log	What it records
Application	Events generated by applications and software.
System	Windows operating system events, drivers, and services.
Security	Security-related events, including logons and account activity.
Setup	Windows installation and configuration events.
Windows PowerShell	PowerShell activity and operational events.

Information displayed

For every event, the program retrieves:

- **Timestamp:** When the event occurred.
- **Event ID:** The identification number assigned to the event.
- **Source:** The application, service, or Windows component responsible for the event.
- **Severity:** Error, Warning, Information, Audit Success, or Audit Failure.
- **Computer Name:** The computer that generated the event.

The script also includes the log name in the CSV file so you can identify which Windows log each event originated from.

---

4. Example program output

When you run the program in Spyder, you should see something similar to this:

```text
==============================================
          WINDOWS EVENT LOG ANALYZER
==============================================

Select an event log to analyze:

1. Application
2. System
3. Security
4. Setup
5. Windows PowerShell
```

```
6. All Event Logs

==============================================

Enter your selection (1-6): 2

Analyzing System Log...
------------------------------------------------------------

Timestamp: 2026-11-10 18:30:15
Event ID: 7040
Source: Service Control Manager
Severity: Information
Computer Name: DESKTOP

------------------------------------------------------------

Successfully processed System Log.
Total events found: 125

==============================================
             EXPORT COMPLETE
==============================================

File saved as: Event_Log_Report_20261110_183015.csv
Total records exported: 125

Event Log Analysis Complete!
```

The output above is an illustration. Your actual results will depend on your computer's event logs.

---

5. Where will the CSV file be saved?

The CSV file will be saved in Spyder's current working directory.

You can find the current directory in Spyder's file explorer or by checking the working directory displayed in its interface.

You can open the CSV file using:

- Microsoft Excel
- Python Pandas
- Notepad
- Other spreadsheet applications

The CSV will contain columns similar to these:

Log Name	Timestamp	Event ID	Source	Severity	Computer Name
System	2026-11-10 18:30:15	7040	Service Control Manager	Information	DESKTOP
Application	2026-11-10 18:32:10	1000	Application Error	Error	DESKTOP
Security	2026-11-10 18:35:20	4624	Microsoft Windows Security Auditing	Audit Success	DESKTOP

These are illustrative records, not actual results.

---

6. Important: Running Spyder as Administrator

One thing to watch out for is the **Security event log**.

Windows restricts access to certain event logs, particularly Security. If you receive an access-denied error:

1. Close Spyder.
2. Find Spyder in your Windows Start Menu.
3. Right-click Spyder.
4. Select **Run as administrator**.
5. Open your Python script again.
6. Run the program.

The script is designed to continue processing the remaining logs if one cannot be accessed, so an error with Security should not prevent the other four logs from being analyzed.

Assignment requirements checklist

Assignment Requirement	Implemented
Read Application log	✅
Read System log	✅
Read Security log	✅
Read Setup log	✅
Read Windows PowerShell log	✅
Select one individual log	✅

Assignment Requirement	Implemented
Select all five logs	✅
Display event timestamp	✅
Display Event ID	✅
Display source	✅
Display severity	✅
Display computer name	✅
Export results to CSV	✅
Continue if a log cannot be accessed	✅
Compatible with Python 3.11	✅
Designed for Windows and Spyder	✅

One important distinction: This script reads the traditional Windows Event Log interface through `pywin32`, which is appropriate for the assignment. It does not retrieve every possible event channel available through the newer Windows Event Log system.